Why Genie Code answers correctly and the Skills MCP does not

A protocol-level comparison of two ways to deliver the same skills to the same LLM

Author: Cursor (Composer 2.5) — analysis prepared for the eToro Data Platform team Date: 2026-06-03 — v2 (rewritten after primary-source verification)

Executive summary

The same LLM family, reading the same `SKILL.md` files, answers business-definition questions correctly when invoked through Databricks Genie Code and approximates them when invoked through our custom Skills MCP from external clients (Claude in coworker mode, Cursor, Claude Desktop).

After investigating both systems against primary sources, the gap is **not** a content failure in the skills. It is a **protocol delta** between Genie Code's published, Anthropic-aligned skill-loading contract and our MCP's bespoke search-and-dump retrieval. Two architectural pieces that Genie Code has, the MCP does not:

1. **Workspace-level imperative routing instructions auto-loaded as system context.** Authored, validated, and shipped by `databricks/data-rules`. Reaches the Genie Code agent every interaction; never reaches external MCP clients.
2. **Progressive disclosure of skills** per the Anthropic Agent Skills specification — discovery (name + description), then activation (one full skill), then execution. Genie Code implements this. Our MCP implements a different protocol: top-K hub bodies plus all nested sub-skill bodies returned in a single tool call.

The corpus is fine. The wrapper is the bottleneck. Aligning the MCP with the protocol Databricks and Anthropic already published is the single highest-leverage fix.

The empirical observation

A user asked an LLM (Claude in coworker mode, via the Skills MCP): "how many funded accounts today?". The LLM correctly called `skills_find_skills`, received `domain-revenue-and-fees` and `domain-customer-and-identity` as candidates, and produced:

```
SELECT COUNT(DISTINCT RealCID) FROM bi_db_ddr_fact_revenue_generating_actions WHERE DateID
```

That counts customers who *generated revenue today* — not the canonical eToro definition of a funded account (`equity > 0` and past the First Time Funded milestone).

The same question, asked of Databricks Genie Code on the same workspace with access to the same UC and the same skills directory, produces a query grounded on

`main.etoro_kpi_prep.gold_de_user_dim_ddr_customer_dailystatus_scd` with `IsFunded = 1` — the correct path.

Same LLM. Same skills with same nesting. Same UC. Different wrapper. Different answer.

What Genie Code actually does

Verified from Databricks's published documentation (`docs.databricks.com/aws/en/genie-code/instructions`, `docs.databricks.com/aws/en/genie-code/skills`) and the official Agent Skills specification at `agentskills.io`.

1. Workspace instructions auto-load as system context

Per the AWS Databricks docs:

> "Workspace instructions are configured by your workspace admin and provide more context to Genie Code to help it follow guidelines and operate more efficiently in your workspace... Genie Code automatically picks up the new workspace instructions the next time a user interacts with it."

The file lives at `/Workspace/.assistant_workspace_instructions.md`, capped at 4,000 characters, and is loaded into the agent's context on every interaction. No tool call is required.

2. Skills follow Anthropic's "progressive disclosure" three-stage protocol

Per `agentskills.io`:

> "Agents load skills through progressive disclosure, in three stages: 1. **Discovery**: At startup, agents load only the name and description of each available skill, just enough to know when it might be relevant. 2. **Activation**: When a task matches a skill's description, the agent reads the full SKILL.md instructions into context. 3. **Execution**: The agent follows the instructions, optionally executing bundled code or loading referenced files as needed."

Genie Code implements this protocol: skills live at `/Workspace/.assistant/skills/<skill-stem>/SKILL.md` (workspace-wide) or `/Users/{username}/.assistant/skills/...` (per-user). The agent loads name + description at startup, activates ONE matching skill at a time, and only then reads the full body.

3. Auto-discovery of `AGENTS.md` and `CLAUDE.md`

Per the AWS Databricks docs, Genie Code "automatically discovers and reads `AGENTS.md` and `CLAUDE.md` up the directory tree" — additional layered context that requires no explicit tool calls.

What our MCP actually does

Verified from the source at `databricks/skills-mcp/databricks-skills-mcp/server/`.

1. Workspace instructions never reach external MCP clients

The gateway has a `SYSTEM_HINT` defined in `databricks-mcp-gateway/server/instructions.py`. Its docstring openly admits:

`` `4:9:databricks/skills-mcp/databricks-mcp-gateway/server/instructions.py **HTTP transport delivery gap:** Anthropic Claude clients (Desktop, claude.ai, Cursor over HTTP) silently drop the MCP `instructions` field (see anthropics/claude-code#41834). Tool descriptions from :mod:`server.middleware` (:class:`ToolDescriptionRewriterMiddleware`) and `databricks-skills-mcp` find_skills are the primary contract for those clients. This file is **defense-in-depth** for stdio / clients that do deliver `instructions``.

```
The `.assistant_workspace_instructions.md` content — the imperative routing rules the Data

### 2. `find_skills` is search-and-dump, not progressive disclosure

In production (`SKILLS_SUB_PASS_ENABLED: "true"` per `app.yaml`), `find_skills` returns:

```json
{
  "skills": [
    {
      "id": "domain-revenue-and-fees",
      "score": 0.82,
      "body_markdown": "<~30 KB hub prose>",
      "sub_skills": [ "<full bodies of all ~6 children, ~120 KB>" ],
      "matched_sub_skills": [ "<top-1 child by query relevance>" ],
      "effective_score": 0.84
    },
    {
      "id": "domain-customer-and-identity",
      "score": 0.78,
      "body_markdown": "<~33 KB hub prose, 283 lines>",
      "sub_skills": [ "<full bodies of all 8 children, ~200 KB>" ],
      "matched_sub_skills": [ "<top-1 child>" ],
      "effective_score": 0.81
    }
  ]
}
```
```

The agent receives top-K hubs, each with full body and all nested sub-skill bodies, in one shot. The Agent Skills spec's three stages are collapsed into one. The agent is then expected to navigate that response and decide which artifact to ground on. **The protocol expects the LLM to do work that Genie Code's protocol does for it.**

The smoking gun — the missing routing prompt already exists

The eToro Data Platform team has *already authored* the equivalent of `.assistant_workspace_instructions.md` and ships it via CD to `/Workspace/.assistant_workspace_instructions.md`. The README at `databricks/data-rules/README.md`:

> "CI/CD-managed source for the Databricks BI workspace's `.assistant_workspace_instructions.md` — the directives that govern how the Databricks Assistant behaves (skill lookup, table-search defaults, FTD routing, etc.)."

Its content includes the exact imperative routing that should resolve the funded-accounts question:

``` ``30:33:databricks/data-rules/.assistant_workspace_instructions.md - Treat **Funded** as equity > 0 AND past the First Time Funded (FTF) milestone (deposit + V3 verification + first action). - For Funded population queries, use main.etoro_kpi_prep.gold_de_user_dim_ddr_customer_dailystatus_scd with IsFunded = 1. - Treat **FTD** (First Time Deposit) as the customer's first deposit on the platform. - For FTD population queries, use main.etoro_kpi.ftd_funnel_v with FirstTimeDeposit_Date IS NOT NULL`.
```

And:

```
```36:39:databricks/data-rules/.assistant_workspace_instructions.md
- For questions about funded, active trader, portfolio only, or balance only, load the cus
- For questions about FTD, registration, verification levels, or conversion funnel, load t
- For questions about revenue, fees, commissions, or trading revenue, load the revenue-bus
- When the intended concept is unclear between Funded, FTD, and revenue, ask the user for
```

The linter at `databricks/data-rules/scripts/lint_rules.py` enforces the imperative form: `no_weasel_words` bans `sometimes / usually / maybe / try to / might / probably / as needed / if possible`; `no_first_second_person` bans `I will / you should / we recommend` ("Rules are issued to the agent, not advice for humans").

This document is exactly what the MCP needs to deliver, exists today, and is already validated. The Genie Code agent reads it and behaves correctly. External MCP clients never see it.

```

Mechanism delta — the clean comparison

Aspect	Genie Code (native)	Skills MCP (external)
System-level routing	`.assistant_workspace_instructions.md` auto-loads as system context, every interaction. 4,000-char cap. Imperative form, lint-enforced.	The equivalent file is not delivered. The MCP `instructions` field is silently dropped by Anthropic clients (per `instructions.py` line 4).
Skill discovery stage	Loads only name + description from each `SKILL.md` at startup. ~50–200 tokens per skill.	`list_skills` exists but is debug-only. `find_skills` jumps straight to bulk content retrieval.
Skill activation stage	Reads full body of ONE skill matched to the question.	Returns top-K hubs (default 5), each with full body, plus all nested sub-skill bodies. ~200 KB+ per response.
Context layering	Workspace instructions + AGENTS.md + CLAUDE.md auto-discovered up the tree.	Single tool-call response; no layered context channel that survives HTTP.
What the LLM sees for "funded"	Workspace instructions tell it which skill to load + which table + the precise definition. Then it activates that one skill.	Multi-hub blob, hundreds of KB of competing prose. Top-level routing rule is missing. The `IsFunded` definition is buried inside `domain-customer-and-identity.sub_skills[6].body_markdown` — present, not salient.
Decisions the LLM has to make	One: ground SQL in the activated skill.	Three: which hub? which sub-skill? do I drill further or ground here?

The corpus content is identical in both paths. **The corpus delivery is fundamentally different.**

Which earlier MCP-side findings still hold

After verification, the diagnosis splits cleanly into "still valid (worth fixing)" and "reframed (subordinate to the protocol fix)":

Still valid — these are real MCP defects

Finding	Location	Status
`effective_score` bubbles the hub but not the matched child to top-level	`tools.py` lines 343–354	Valid. Should hoist child as primary when it outscores the hub.
`SKILLS_SUB_K_DEFAULT = 1` makes the top-1 child a coin flip	`app.yaml`	Valid. Bump to 3 minimum.
`find_skills` tool description teaches one-shot grounding	`tools.py` line 156	Valid. Tool descriptions are one of the few channels that reach HTTP clients.
No banner on `find_skills` to teach the second hop	`middleware.py` lines 284–311	Valid. Add one or restructure into a progressive-disclosure call sequence.
Sub-skill stems not deduped by IDENTITY-003	`validate_skills.py`	Valid. Already exposed by the recent collision incident.

Reframed — subordinate to the protocol fix

The earlier framing of "rewrite hub bodies with imperative STOP blocks" was wrong in emphasis. Hub bodies are not the failure surface — Genie Code reads the same hubs and behaves correctly. Hub authoring tightening is **defensive**, not the leverage move. The leverage move is fixing the wrapper.

The "LLM cognitive bias" section of the v1 report (confidence saturation, path-of-least-resistance, mode-switching gap) is real LLM behaviour, but it activates **conditional on the wrapper**. Genie Code's wrapper does not activate it. Our MCP's wrapper does. The biases are not skills-reading failure modes — they are **MCP-protocol-induced** failure modes.

The fix path — align the MCP with the protocol Databricks already publishes

Tier 1 — protocol parity (largest leverage)

1. **Ship the workspace-instructions file to MCP clients.** Mirror `.assistant_workspace_instructions.md` content into the MCP's response surface. Two delivery channels, both should be used: - **Tool descriptions** (HTTP-safe, primary channel for Cursor / Claude Code / Desktop). Trim the 4,000 char file to its critical routing rules and bake them into `find_skills` + `get_skill` + the gateway banner middleware. - **A new top-level field on** `find_skills`

responses — for example `routing_protocol` — that returns the file content verbatim (or a question-relevant slice) on every call. Survives the HTTP transport because it is data, not metadata.

2. **Implement progressive disclosure.** Restructure `find_skills` and `get_skill` to match the Anthropic Agent Skills three-stage spec: - `find_skills` (discovery) returns ONLY name + description + match score for top-K skills (skills, not hubs — sub-skills are first-class). ~200 tokens per result, not 30 KB. - `get_skill(id)` (activation) returns the full body for one skill at a time. - References within the body load on demand at execution. - This is the Anthropic spec the Databricks Assistant already implements. Aligning here moves us onto a published, stable contract.

Tier 2 — MCP-internal cleanups (compounds with Tier 1)

3. Hoist a matched child to top-level when its score exceeds the hub's by some delta — automatic in the new shape once Tier 1 lands.
4. Bump `SKILLS_SUB_K_DEFAULT` from 1 to 3 in `app.yaml`.
5. Add a `find_skills` banner to `ToolDescriptionRewriterMiddleware` reminding the LLM to drill into matched_sub_skills via `get_skill`.
6. Extend IDENTITY-003 in `validate_skills.py` to dedupe sub-skill stems (closes the collision class your last PR fixed manually).

Tier 3 — defensive content tightening (low priority)

7. Light pass over hub bodies to convert the soft "see X.md" pointers to imperative `get_skill('X')` callouts. Not required for correctness once Tier 1 ships, but useful for stdio clients that DO honour `instructions`.

What to ship this week

Pick **#1 (workspace instructions in tool descriptions + a `routing_protocol` field on `find_skills`)**. It is the single change that lets external MCP clients see what `Genie Code` already sees. The rest of the stack stays untouched, the corpus stays untouched, and the same LLM that gets it right in the Databricks Assistant starts getting it right in Cursor / Claude Code / Desktop.

Concrete first deliverable: a one-paragraph addition to the `find_skills` response shape that returns the relevant slice of `.assistant_workspace_instructions.md`, and a banner update on the `find_skills` tool description that says "consult `routing_protocol` first; load `get_skill(id)` for activation". Days, not weeks. Authored content already exists.

A note on methodology

The first version of this report (issued an hour earlier on the same day) framed the diagnosis as "LLMs over-anchor on prominent hub bodies and ignore prose pointers to sub-skills" and proposed re-authoring all 13 hubs with imperative STOP blocks. That framing was wrong on emphasis — though most of the

MCP-side findings remained valid. The reframing came from a single observation by the user: that Databricks Genie Code, reading the same skill files with the same nesting, gets the answer right. That observation forced a primary-source re-investigation: reading `databricks/data-rules/.assistant_workspace_instructions.md`, the Databricks docs at `docs.databricks.com/aws/en/genie-code/instructions` and `…/genie-code/skills`, and the Anthropic Agent Skills spec at `agentskills.io`. The diagnosis above is grounded in those sources rather than in inference about how a competing system might be structured.

The general lesson is the one the report itself diagnoses: under pressure to produce a complete-looking answer, an LLM will fill in mechanism details from priors when verification is one search away. The sufficient remedy is the same in both cases — make the verification path cheaper than the approximation path. For Genie Code that means workspace-instructions auto-load and progressive disclosure. For an analyst-LLM writing a diagnostic report, that means primary sources cited, not extrapolation.

Prepared by Cursor (Composer 2.5) on 2026-06-03 (v2). Sources: *`databricks/skills-mcp/databricks-skills-mcp/server/`*, *`databricks/skills-mcp/databricks-mcp-gateway/server/`*, *`databricks/data-rules/.assistant_workspace_instructions.md`*, *`databricks/data-skills/skills/domain-customer-and-identity/`*, *Databricks Genie Code documentation, Anthropic Agent Skills specification at agentskills.io.*